Final Adjustments to C++26 Reflection

Document #:	P3687R0 [Latest] [Status]
Date:	2025-05-15
Project:	Programming Language C++
Audience:	EWG, LEWG

Reply-to:	Wyatt Childers <wcc@edg.com> Dan Katz <dkatz85@bloomberg.net> Daveed Vandevoorde <daveed@edg.com> Ville Voutilainen <ville.voutilainen@gmail.com>

Contents

§ 1 Abstract

We propose two minor design changes to [P2996R12] to ensure its readiness for C++26 and future-proof the evolution of the feature.

First, we propose that “splice template arguments” be removed from P2996 and deferred until C++29: Neither the wording nor the implementation are fully baked, CWG has not (as of this writing) dedicated sufficient time to its review, and Reflection suffers no loss of expressive power from its removal.

Second, in response to edge cases surfaced during CWG review, we elaborate on the need to represent *`using-declaration`*s with Reflection. At this late stage, we offer two means of ameliorating this feature’s absence:

1. Permit a narrow expansion of scope to what is already offered by P2996 by introducing reflections of *`using-declaration`*s to C++26, or

2. Make `^^Cls::mem` ill-formed whenever `mem` names a *`using-declaration`*, thus preserving the syntactic space for C++29.

§ 2 Background

§ 2.1 Removing splice template arguments

§ 2.1.1 Refresher: What are “splice template arguments”?

P2996 proposes *`splice-template-argument`*s that can match any other kind of template argument. For instance, given

```
template <class P1, auto P2, template<typename> class P3>
struct mytemplate;
```

the *`template-id`*

```
mytemplate<[:R1:], [:R2:], [:R3:]>
```

will be valid as long as the constructs represented by `R1`, `R2`, and `R3` match the form of the respective template parameters.

§ 2.1.2 Would removing this mean that I can’t use splices as template arguments?

No. An expression splice (i.e., *`splice-expression`*) can appear as a constant template argument, and a type splice (i.e., *`splice-type-specifier`*) can appear as a type template argument.

```
constexpr auto r1 = ^^int;
constexpr auto r2 = std::meta::reflect_value(42);

std::array<typename [:r1:], [:r2:]> arr;  // OK
```

Splice template arguments, the feature that we here propose to remove from P2996, lets the syntax `[:R:]` uniformly match any kind of template parameter when appearing in a template argument list. So while the above example remains valid, the following become disallowed because (in the absence of “splice template argument” support) unqualified `[:R:]` always parses as an expression:

```
std::array<typename [:r1:], [:r2:]> arr1;
  // still OK
std::array<[:r1:], [:r2:]> arr2;
   // error: reflection of a type 'r1' cannot splice as an expression; did you mean
         'typename [:r1:]'?
  // error: cannot splice 'r1' (a reflection of a type) as an expression
  // error: expression '[:r1:]' does not match template type parameter

template <template <typename T, int V> class Tmp, typename T, int V>
auto make() { return Tmp<T, V>{}; }

constexpr std::meta::info rs[] = {^^std::array, ^^int, std::meta::reflect_value(42)};

auto arr = make<[:rs[0]:], [:rs[1]:], [:rs[2]:]>();
```

```
  // error: expression '[:rs[0]:]' does not match template template parameter 'Tmp'
  // error: expression '[:rs[1]:]' does not match type template parameter 'T'
```

2.1.3 Why defer to C++29?

Consider the following example:

```
template <template <typename> class TCls>
auto tfn1() {
  return TCls<int>{};
}

template <std::meta::info R>
template tfn2() {
  return tfn1<[:R:]>();
}
```

For the *`template-id`* `tfn1<[:R:]>` to be valid in the template definition of `tfn2()`, we must be able to substitute the dependent splice template argument `[:R:]` into `tfn1`. In the parlance of P2996, this entails replacing the *`template-id`* "`TCls<int>`" with the *`splice-specialization-specifier`* "`[:R:]<int>`", and replacing the *`template-name`* "`TCls`" with the *`splice-specifier`* "`[:R:]`".

Since Clang implements template substitution as a tree transformation (i.e., "recursively substitute like things with like things"), the Clang reference implementation of P2996 handles this by adding a new category of `DependentTemplateName` in which a `TemplateName` wraps a `SpliceSpecifier`.

This implementation strategy mirrors the grammar proposed by [P2996R7], in which a *`template-name`* might be a *`splice-specifier`*. Discussion within CWG, however, steered us away from this direction (for good reason: a *`splice-specifier`* is not a "name"). The primary author of the Clang/P2996 reference implementation has not identified a suitable alternative implementation strategy, reinforcing the belief that splice template arguments, as currently specified, *require* a tighter coupling of *`template-name`* and *`splice-specifier`*. As tempting as it is to dismiss this as "just a wording problem", the wording strategy is not easily separable from the implementation strategies that will be adopted by compilers.

Lastly, as of this writing, CWG is yet to invest meaningful time in the review of *`splice-template-argument`*s. The combination of our belief that the wording is insufficient, together with the scrutiny that the feature still requires, leaves us feeling that dropping the feature from P2996 will strengthen P2996's readiness for C++26.

2.1.4 What power is lost from P2996?

Absolutely none. Any *`template-id`* written with *`splice-template-argument`*s as

```
template_name<[:R1:], [:R2:], [:Rs:]...>
```

can be equivalently expressed using `std::meta::substitute`:

```
[:substitute(^^template_name, {R1, R2, Rs...}):]
```

Splitting splice template arguments from the paper, therefore, represents a means of reducing the P2996 surface area that CWG still needs to review, while losing absolutely no expressive power from P2996.

2.1.5 Can we add this later?

Absolutely. For C++26, a template argument of the form `[:R:]` will always be considered a *splice-expression*, which can only match a constant template parameter. In C++29, we can make it well-formed for this syntax to also match type template parameters and template template parameters.

That said, the change will not be entirely non-breaking; consider the following example provided by Tomasz Kamiński:

```
template <auto>     int foo();  // A
template <typename> int foo();  // B

template <std::meta::info R>
auto bar(int) -> decltype(foo<[:R:]>()) { return 1; }  // #1

template <std::meta::info R>
auto bar(...) -> int                    { return 0; }  // #2

constexpr int v = bar<^^int>(1);
```

In the initializer for `v`, `bar` denotes an overload set containing both #1 and #2. Function template argument deduction for #1 will fail, since the substitution of "`^^int`" into "`R`" will form "`[:^^int:]`" - an invalid *splice-expression*. Since this ill-formed expression is within the immediate context, the specialization will merely be discarded from the set of candidate functions. We are left with the specialization derived from #2, within which no such ill-formed expression appears; `v` is initialized to `0`.

But in the presence of splice template arguments, "`[:^^int:]`" is parsed as a *splice-template-argument* rather than an expression. The non-dependent *splice-template-argument* is interpreted as a *splice-type-specifier*, so the `B` overload is selected. Because #1 is a better match than #2, `v` would initialized to `1`.

WG21 has historically found changes to overload resolution to be acceptable, even when it has meant breaking SFINAE-dependent programs. We therefore feel that the suggestion to adopt splice template arguments later rests on solid ground - but it's certainly still worth pausing to understand the implications.

2.1.6 What if we want to keep it?

Risky, but also not the end of the world. The grammar for *template-name* ([temp.names]) will likely have to be extended:

```
template-name:
    identifier
+   splice-specifier
```

Which is not only an invasive change, but one that CWG has judged to be ill-advised. The P2996 authors will have to (again) comb through the corpus of core wording to find what other sections must be modified to account for the change, while adding various "if the *template-name* is an *identifier*" carve

outs for cases where the *template-name* is more “name”-like than “computed entity specifier”-like. It is the opinion of the authors that further substantial core wording homework would neither be beneficial to the prospects of Reflection making C++26 nor to their emotional well-being.

§ 2.2 The necessity of reflecting *using-declaration*s

§ 2.2.1 Motivation

Consider the following example:

```
struct Base { int member = 0; };
struct Derived : private Base { using Base::member; };

Derived d;
auto p1 = d.member;  // OK
auto p2 = d.[:^^Derived::member:];   // error
```

Consider also the following assertions, which all hold with [P2996R12]:

```
#include <meta>

struct Base {
  protected: int member;
  friend void fn();
};

struct Derived : private Base {
  using Base::member;
};

void fn() {
  constexpr auto ctx = std::meta::access_context::unprivileged();
  static_assert(!is_accessible(^^Base::member, ctx));
  static_assert(!is_accessible(^^Derived::member, ctx));
  static_assert(!is_accessible(^^Base::member, ctx.via(^^Derived)));
  static_assert(!is_accessible(^^Derived::member, ctx.via(^^Derived)));
}

auto p = &Derived::member;  // no problem
```

In particular, there is currently no way to observe

1. That the *using-declaration* naming `Base::member` exists in the scope of `Derived` or
2. That `Base::member` can be accessibly named in the scope of `Derived`.

The first point implies restrictions for e.g., debug tools that may wish to format a representation of a class type. The second implies a limited view of which members are accessible within a class.

§ 2.2.2 What? Why?!

In both examples above, the expression `^^Derived::member` represents the entity `Base::member` because *using-declaration*s are transparently replaced with the declarations that they name during lookup

([basic.lookup]/3). In the first example, the expression

```
d.[:^^Derived::member:]
```

is ill-formed because the type of the left-hand-side (i.e., `Derived &`) cannot be converted to the "naming class" of the right-hand-side (i.e., `Base`) ([class.access.base]/6); since `^^Derived::member` represents `Base::member`, this is the moral equivalent of writing `d.Base::member`. Similarly in the second example, the expression

```
is_accessible(^^Derived::member, ctx.via(^^Derived))
```

asks whether the class member `Base::member` is accessible from a point in the global scope when named in the class `Derived`. The algorithm from [class.access.base]/5 tells us that this is not the case, which mirrors the fact that writing `Derived::Base::member` is ill-formed. If we want to instead determine whether `Derived::member` is well-formed, we must instead ask whether the *`using-declaration`* is accessible in `Derived`, a question which P2996 lacks the machinery to model.

§ 2.2.3 Why was this not in P2996R0 or P2996R1 or P2996R2 or …?

During early revisions of P2996, the authors did not seek to integrate Reflection with Access Control. Around the time of the Wrocław meeting, it became more clear that a robust story for observing the accessibility of class members would be required to achieve consensus for the design. The direction proposed by [P3547R1] received strong support in Hagenberg, and was thereafter merged into [P2996R10].

*`using-declarator`*s, as currently specified, are not a natural target for Reflection: Since they introduce only names (rather than entities or class members), there is "nothing to reflect" (in this regard, they are more similar (pre-P2996) to type aliases and namespace aliases than to e.g., non-static data members). They nevertheless play an important role in Access Control: An otherwise inaccessible member of a base class might be accessible when named through a *`using-declarator`* declared in a derived class (see [class.access.general]/4).

The surprising behavior exhibited by the first example, in which the "surprising answer" is obtained using only the reflection and splicing operators, was only very recently uncovered by a careful review of [expr.ref] and [class.access] during CWG review.

§ 2.2.4 How should this be handled?

What follows is a small extension on top of what is proposed by P2996. We provide it as an elaboration of how we believe Reflection should interact with *`using-declaration`*s in an unspecified future delivery vehicle of the C++ Standard.

— First, we propose modifying the core wording around *`using-declarator`*s such that they *do* introduce entities (analogous to existing changes in P2996 for type aliases and namespace aliases). In particular a declaration

```
class Cls {
  using Scope::id;
};
```

introduces a class (or namespace) member known as an *entity proxy* for each entity whose declaration is named by `Scope::id` (note: several such members may be introduced if `Scope::id` denotes an overload set). The "underlying entity" of each such entity proxy is one of the entities whose declarations were named by `Scope::id`. Note that this is scarcely more than a technical wording change to make sure there is "something to have a reflection of".

— Next, we propose that a *`reflect-expression`* `^^Cls::id` whose operand names a unique entity proxy, **represents said entity proxy**, and not its underlying entity. This gives a means of observing the properties of the entity proxy (e.g., accessibility, parent class) apart from the properties of its underlying entity.

— Since entity proxies are members of classes and namespaces, we propose that the reflections returned by `members_of` also include entity proxies (i.e., in formal P2996 parlance: entity proxies that are members of a class or namespace `Q` are *`Q`-members-of-representable*).

— If lookup for `Cls::id` finds multiple declarations named `id` in the scope of `Cls`, the *`reflect-expression`* `^^Cls::id` is ambiguous and ill-formed, even if all such declarations denote the same underlying entity.

```
class A { void fn(); };
class B : virtual A { using A::fn; };
class C : virtual A { using A::fn; };
class D : B, C {};

auto p = &D::fn;  // OK
constexpr auto r = ^^D::fn;  // ambiguous, ill-formed
```

In a more distant future, it may be possible to let such expressions represent *all* entities named by the ambiguous lookup; this direction would take advantage of how Reflection separates the lookup of a name from its point of use. Such "lookup sets" could be a powerful extension that facilitates (and generalizes) overload sets.

— Existing P2996 metafunctions would be updated to observe properties of entity proxies (e.g., `is_public`, `parent_of`, `is_class_member`), and a new `is_entity_proxy` function would be introduced to recognize such reflections.

— Lastly, the P2996 metafunction `std::meta::dealias` should map a reflection of an entity proxy to a reflection of its underlying entity. Since the phrase "underlying entity" is a Word of Power introduced by P2996 that applies equally well to aliases and entity proxies, we propose renaming `std::meta::dealias` to `std::meta::underlying_entity_of`. A separate `std::meta::proxied_entity_of` function returns a reflection of the entity directly named by an entity proxy without "unwrapping all layers" to reach the underlying entity.

```
class B { using Alias = int; };
class D : B { using B::Alias; };

static_assert(is_type_alias(^^B::Alias));
static_assert(is_entity_proxy(^^D::Alias));
static_assert(^^B::Alias != ^^D::Alias);
static_assert(proxied_entity_of(^^D::Alias) == ^^B::Alias);
```

```
static_assert(underlying_entity_of(^^D::Alias) == ^^int);
static_assert(underlying_entity_of(^^B::Alias)                                    ==
        underlying_entity_of(^^D::Alias));
```

2.2.5 Can this all wait until C++29?

Given the following code:

```
struct A { protected: int m; };
struct B : private A { using A::m; };

constexpr auto rm = ^^B::m;
```

We must decide now whether `^^B::m`

- represents the member `A::m` that is a member of `A` (**P2996 status quo**),
- represents a member `B::m` that is a member of `B` and whose underlying entity is `A::m` (as elaborated above), or
- is ill-formed.

That decision will be observable to programs in several ways (e.g., whether `parent_of(rm)` is `^^B` or `^^A`, the result of `is_public(rm)`, etc). If we would prefer that `^^B::m` instead represent a member of `B`, we should either adopt that change now or make the expression ill-formed entirely to reserve space for that change in C++29. Either way, we should make that change for C++26.

2.2.6 What else can we do with entity proxies?

Although the "generative metaprogramming" facilities directly provided by P2996 will be narrow (e.g., `define_aggregate`), it will be possible to inspect the members and properties of a class, compute a string containing a well-formed C++ class definition derived from that class, render that string to `stdout`, and write the resulting code to a file or a pipe to another process (e.g., a C++ compiler). Such pipelines will benefit from being able to inspect *`using-declarator`*s, as the transformed source code will more exactly correspond to the class from which it was transformed.

As an aside, the proposed direction ought to make it possible to implement a `class_lookup` library function that accurately models the algorithm specified in [class.member.lookup], which is pretty sweet. I'm sure we can think of something fun to do with that.

3 Implementation experience

Our proposal for "entity proxies" is entirely implemented in Bloomberg's Clang/P2996 fork. Passing either `-fentity-proxy-reflection` or `-freflection-latest` (the latter being a "catch all" for all implemented reflection and reflection-adjacent papers) enables the feature. An example on Godbolt can be found here.

Splice template arguments are implemented in Clang/P2996, but the effort to refactor and decouple their implementation from "template names" motivated this proposal's recommendation that they for now be removed.

4 Suggested polls

The authors submit the following polls to EWG for their consideration.

1. Remove support for *splice-template-argument*s from P2996R12.

SF	F	N	A	SA

2a. Add reflection of “entity proxies” to P2996 and forward to CWG for inclusion in C++26.

SF	F	N	A	SA

2b. Make `^^id` ill-formed when `id` names a *using-declaration* and forward to CWG for inclusion in C++26.

(Only suggested if [2a] does not find consensus)

SF	F	N	A	SA

5 Proposed wording

5.1 Poll 1: Remove *splice-template-argument*s from P2996

[Drafting note: All wording assumes P2996R12.]

[temp.param] Template parameters

Strike the P2996 changes to the grammar at the beginning of [temp.param]:

```
  type-tt-parameter-default:
      nested-name-specifier_opt template-name
      nested-name-specifier template template-name
-     splice-template-argument

  variable-tt-parameter:
      template-head auto ..._opt $identifier_opt
           template-head auto identifier_opt = nested-name-specifier_opt
        template-name
-     template-head auto identifier_opt = splice-template-argument

  concept-tt-parameter:
      template < template-parameter-list > concept ..._opt identifier_opt
```

```
          template < template-parameter-list > concept identifier_opt =
        nested-name-specifier_opt template-name
-       template < template-parameter-list > concept identifier_opt =
        splice-template-argument
```

§ [temp.names] Names of template specializations

Strike the P2996 changes to the grammar for *template-argument*:

```
  template-argument:
      constant-expression
      type-id
      nested-name-specifier_opt template-name
      nested-name-specifier_opt template template-name
-     splice-template-argument

- splice-template-argument:
-     splice-specifier
```

§ [temp.arg.general] General

Strike the sentence from paragraph 1 pertaining to splice template arguments from P2996:

1 The type and form of each *template-argument* specified in a *template-id* or in a *splice-specialization-specifier* shall match the type and form specified for the corresponding parameter declared by the template in its *template-parameter-list*. ~~A *template-argument* that is a splice template argument is considered to match the form specified for the corresponding template parameter.~~ When the parameter declared by the template is a template parameter pack, it will correspond to zero or more *template-argument*s.

Revert the changes to paragraph 3 from P2996:

3 ~~A *template-argument* of the form *splice-specifier* is interpreted as a *splice-template-argument*. For any other~~ In a *template-argument*, an ambiguity between a *type-id* and an expression is resolved to a *type-id*, regardless of the form of the corresponding *template-parameter*.

§ [temp.arg.type] Template type arguments

Strike the changes to [temp.arg.type] from P2996:

1 A *template-argument* for a type template parameter shall ~~either~~ be a *type-id* ~~or a~~

~~*splice-template-argument* whose *splice-specifier* designates a type~~.

§ [temp.arg.template] Template template arguments

Strike the changes to [temp.arg.template] from P2996:

1 A *template-argument* for a template template parameter shall ~~either~~ be the name of a template ~~or a *splice-template-argument*~~. For a *type-tt-parameter*, the name ~~or *splice-template-argument*~~ shall denote ~~designate~~ a class template or alias template. For a *variable-tt-parameter*, the name ~~or *splice-template-argument*~~ shall denote ~~designate~~ a variable template. For a *concept-tt-parameter*, the name ~~or *splice-template-argument*~~ shall denote ~~designate~~ a concept. Only primary templates are considered when matching the template argument with the corresponding parameter; partial specializations are not considered even if their parameter lists match that of the template template parameter.

§ [temp.dep.temp] Dependent template arguments

Strike the paragraph covering splice template arguments from P2996.

5 A template *template-parameter* is dependent if it names a *template-parameter* or if its terminal name is dependent.

5+ ~~A splice template argument is dependent if its *splice-specifier* is dependent.~~

§ 5.2 Poll 2a: Augment P2996 with reflection of entity proxies

[Drafting note: All wording assumes P2996R12.

The general approach starts from [namespace.udecl]: whereas a *using-declarator* previously "named" or "nominated" or "referred" to a set of declarations, it is now said to *proxy* those declarations. A *using-declarator* then introduces an *entity proxy* for each entity introduced by one of the declarations that it proxies. The *using-declarator* names the entity proxies, but can still be said to "proxy" the found declarations.

Entity proxies are entities; they can be class members, so *using-declaration*s now introduce class members. The existing [class.access.base] definition for when a "member is accessible when designated in a class from a point" now works much more cleanly for *using-declaration*s.

Most other core wording changes are just preferring the very "proxies" to e.g., "names".]

§ [basic.pre] Preamble

Add "entity proxies" to the list of entities in pargraph 8.

8 An *entity* is a variable, structured binding, result binding, function, enumerator, type, type alias, non-static data member, bit-field, template, namespace, namespace alias, entity proxy, template parameter, function parameter, or *`init-capture`*. The *underlying entity* of an entity is that entity unless otherwise specified. A name *denotes* the underlying entity of the entity declared by each declaration that introduces the name.

[*Note 1:* Type aliases, ~~and~~ namespace aliases, and entity proxies have underying entities that are distinct from themselves. — *end note*]

§ [basic.lookup] General

Change “named” to “proxied” in paragraph 3.

3 A *single search* in a scope *`S`* for a name *`N`* from a program point *`P`* finds all declarations that precede *`P`* to which any name that is the same as *`N`* ([basic.pre]) is bound in *`S`*. If any such declaration is a *`using-declarator`* whose terminal name ([expr.prim.id.unqual]) is not dependent ([temp.dep.type]), it is replaced by the declarations proxied ~~named~~ by the *`using-declarator`* ([namespace.udecl]).

§ [class.qual] Class members

Change “names” to “proxies” in paragraph 1.2:

(1.2) — if *`N`* is dependent and is the terminal name of a *`using-declarator`* ([namespace.udecl]) that ~~names~~ proxies a constructor,

§ [basic.fundamental] Fundamental types

Add a new bullet to the list of constructs that a reflection can represent.

16 The types denoted by *`cv`* `std::nullptr_t` are distrinct types. […]

x A value of type `std::meta::info` is called a *reflection*. There exists a unique *null reflection*; every other reflection is a representation of

(x.1) — […]

(x.16) — a namespaace alias ([namespace.alias]),

(x.17) — a namespace ([basic.namespace.general]),

(x.17+) — an entity proxy ([namespace.udecl]),

(x.18) — a direct base class relationship ([class.derived.general]), or

(x.19) — a data member description ([class.mem.general]).

A reflection is said to *represent* the corresponding construct. […]

§ [expr.reflect] The reflection operator

Extend paragraph 5 to allow a *`reflect-expression`* to represent an entity proxy.

5 If a *`reflect-expression R`* matches the form `^^` *`qualified-reflection-name`*, it is interpreted as such and its representation is determined as follows:

(5.*) — If the *`identifier`* names a unique entity proxy ([namespace.udecl]), *`R`* represents that entity proxy. If the *`identifier`* names multiple entity proxies, *`R`* is ill-formed.

(5.1) — Otherwise, ~~I~~if the *`identifier`* is a *`namespace-name`* that names a namespace alias ([namespace.alias]), *`R`* represents that namespace alias. For any other *`namespace-name`*, *`R`* represents the denoted namespace.

(5.2) — […]

§ [namespace.udecl] The `using` declaration

Modify paragraph 1 such that a *`using-declarator`* introduces an entity called an “entity proxy”.

1 The component names of a *`using-declarator`* are those of its *`nested-name-specifier`* and *`unqualified-id`*. Each *`using-declarator`* in a *`using-declaration`*[84] introduces an *entity proxy* corresponding to each entity denoted by the *`using-declarator`* ~~names the set of declarations found by lookup ([basic.lookup.qual])~~.

Each *`using-declarator`* in a *`using-declaration`*[84] ~~names~~ *proxies* the set of declarations found by lookup ([basic.lookup.qual]) for the *`using-declarator`*, except the class and enumeration declarations that would be discarded are merely ignored when checking for ambiguity ([basic.lookup]), conversion function templates with a dependent return type are ignored, and certain functions are hidden as described below. If the terminal name of the *`using-declarator`* is dependent ([temp.dep.type]), the *`using-declarator`* is considered to ~~name~~ proxy a constructor if and only if the *`nested-name-specifier`* has a terminal name that is the same as the *`unqualified-id`*. If the lookup in any instantiation finds that a *`using-declarator`* that is not considered to ~~name~~ proxy a constructor does do so, or that a *`using-declarator`* that is considered to ~~name~~ proxy a constructor does not, the program is ill-formed.

Each *`using-declarator`* that proxies a declaration of an entity *`E`* declares a unique *entity proxy* that is said to *proxy* *`E`*. An entity proxy that proxies an entity *`E`* has the same underlying entity as *`E`*.

§ [module.interface] Export declaration

Adopt the language of “proxying” in paragraph 5.

5 If an exported declaration is a *using-declaration* ([namespace.udecl]) and is not within a header unit, the underlying entity of each entity proxy thereby introduced ~~to which all of the *using-declarator*s ultimately refer~~ (if any) shall have been introduced with a name having external linkage.

§ [class.mem.general] General

Remove *using-declarator*s from the list of declarations that do not introduce a class member in paragraph 4.

4 A *member-declaration* does not declare new members of the class if it is

(4.1) — a friend declaration ([class.friend]),

(4.2) — a *deduction-guide* ([temp.deduct.guide]),

(4.3) — a *template-declaration* whose *declaration* is one of the above,

(4.4) — a *static_assert-declaration*, or

(4.5) — ~~a *using-declaration* ([namespace.udecl]), or~~

(4.6) — an *empty-declaration*.

§ [class.copy.assign] Copy/move assignment operator

Use “proxies” in the note that follows paragraph 8.

[*Note 5:* A *using-declaration* in a derived class `C` that ~~names~~ proxies an assignment operator from a base class never suppressess the implicit declaration of an assignment operator of `C`, even if the base class assignment oeprator would be a copy or move assignmentoperator if declared as a member of `C`. — *end note*]

§ [class.access.general] General

Refer to the declarations “proxies” by *using-declarator*s, rather than those “named” (i.e., the entity proxies), in paragraph 4.

Access control is applied uniformly to declarations and expressions.

[*Note 2:* Access control applies to members nominated by friend declarations ([class.friend]) and *using-declaration*s ([namespace.udecl]). — *end note*]

When a *using-declarator* is named, access control applies to the entity proxies nominated by it, not to the proxied declarations that replace it.

§ [over.match.funcs.general] General

Change “nominated” to “proxied” in paragraph 4.

4 For implicit object member functions, the type of the implicit object parameter is […].

[*Example 1:* For a `const` member function of class `X`, the extra parameter is assumed to have type “lvalue reference to `const X`”. — *end example*]

For conversion functions that are implicit object member functions, […]. For non-conversion functions that are implicit object member functions ~~nominated~~ proxied by a *using-declaration* in a derived class, the function is considered to be a member of the derived class for the purpose of defining the type of the implicit object parameter. For static member functions, […].

§ [temp.spec.partial.general] General

Change “refers” to “proxies” in the note that follows paragraph 7.

[*Note 1:* One consequence is that a *using-declaration* which ~~refers to~~ proxies a class template does not restrict the set of partial specializations that are found through the *using-declaration*. — *end note*]

[Drafting note: On the library side, we do not want entity proxies to “act like” their underlying entity (e.g., `members_of` should not enumerate the members of the underlying entity of an entity proxy). For that reason, `underlying_entity_of(r)` is too blunt of a tool for specifying such functions.

We instead introduce an exposition-only *DEALIAS* function, which maps a reflections of entity proxies to the null reflection. This causes entity proxies to fail the preconditions for functions that should not apply to them, and makes for useful wrapper around `underlying_entity_of` for purposes of specification.

Other than adding new functions and renaming `dealias` to `underlying_entity_of`, the only other noteworthy change is that `members_of` returns also entity proxies.]

§ [meta.reflection.synop]

Add the `is_entity_proxy` and `proxied_entity_of` functions.

Header `<meta>` synopsis

```
#include <initializer_list>

namespace std::meta {
  [...]
  consteval bool is_enumerable_type(info r);

  consteval bool is_variable(info r);
  consteval bool is_type(info r);
  consteval bool is_namespace(info r);
  consteval bool is_type_alias(info r);
  consteval bool is_namespace_alias(info r);
  consteval bool is_entity_proxy(info r);

  consteval bool is_function(info r);
  [...]
  consteval bool has_default_member_initializer(info r);

  consteval bool has_parent(info r);
  consteval info parent_of(info r);

  consteval info dealiasunderlying_entity_of(info r);
  consteval info proxied_entity_of(info r);

  consteval  bool has_template_arguments(info r);
  [...]
}
```

Change `dealias` to `underlying_entity_of` in the second note that follows paragraph 2.

> [*Note 2:* The behavior of many of the functions specified in namespace `std::meta` have semantics that can be affected by the completness of class types represented by reflection values. For such functions, for any reflection `r` such that `dealiasunderlying_entity_of(r)` represents a specialization of a templated class with a reachable definition, the specialization is implicitly instantiated ([temp.inst]). — *end note*]

§ [meta.reflection.operators] Operator representations

Modify `operator_of` to account for entity proxies:

```
  consteval operators operator_of(info r);
```

2 *Constant When*: `underlying_entity_of(r)` represents an operator function or operator function template.

§ [meta.reflection.names]

Modify `has_identifier` to account for entity proxies:

```
consteval bool has_identifier(info r);
```

Returns:

(1.1) — […]

(1.9) — Otherwise, if `r` represents an enumerator, non-static data member, namespace, ~~or~~ namespace alias, or entity proxy, then `true`.

(1.10) — Otherwise, […]

§ [meta.reflection.queries]

Introduce a metasyntactic function following *`has-type`* to "unwrap" a reflection to its underlying entity, while excluding entity proxies.

```
consteval bool has-type(info r); // exposition only
```

Returns: `true` if `r` represents a value, object, variable, function that is not a constructor or destructor, enumerator, non-static data member, unnamed-bit-field, direct base class relationship, or data member description. Otherwise, `false`.

```
consteval info DEALIAS(info r); // exposition only
```

* *Returns*: `underlying_entity_of(r)` if `r` represents an entity that is not an entity proxy. Otherwise, the null reflection.

Disallow entity proxies from being used with `value_of`.

```
consteval info value_of(info r);
```

7 Let *`R`* be a constant expression of type `info` such that *`R`* `==` *`DEALIAS`*`(r)` is `true`.

Specify `is_const` and `is_volatile` in terms of *`DEALIAS`*.

```
consteval bool is_const(info r);
consteval bool is_volatile(info r);
```

19 Let *`T`* be `type_of(r)` if *`has-type`*`(r)` is `true`. Otherwise, let *`T`* be ~~`dealias`~~*`DEALIAS`*`(r)`.

Also specify `is_lvalue_reference_qualified` and `is_rvalue_reference_qualified` in terms of *`DEALIAS`*.

```
consteval bool is_lvalue_reference_qualified(info r);
consteval bool is_rvalue_reference_qualified(info r);
```

Let *T* be `type_of(r)` if *`has-type`*`(r)` is `true`. Otherwise, let *T* be ~~`dealias`~~*`DEALIAS`*`(r)`.

Change `dealias` to *`DEALIAS`* for `is_complete_type`:

Returns: `true` if `is_type(r)` is `true` and there is some point in the evaluation context from which the type represented by ~~`dealias`~~*`DEALIAS`*`(r)` is not an incomplete type ([basic.types]). Otherwise, `false`.

Account for entity proxies in the specification of `is_enumerable_type`.

```
consteval bool is_enumerable_type(info r);
```

A type *T* is *enumerable* from a point *P* if either […]

Returns: `true` if ~~`dealias`~~*`DEALIAS`*`(r)` represents a type that is enumerable from some point in the evaluation context. Otherwise, `false`.

Use *`DEALIAS`* in `is_type` and `is_namespace`. Specify the behavior of `is_entity_proxy` following the specification of `is_namespace_alias`:

```
consteval bool is_type(info r);
consteval bool is_namespace(info r);
```

Returns: `true` if ~~`dealias`~~*`DEALIAS`*`(r)` represents a type or namespace, respectively. Otherwise, `false`.

```
consteval bool is_type_alias(info r);
consteval bool is_namespace_alias(info r);
```

Returns: `true` if `r` represents a type alias or a namespace alias, respectively [*Note 4:* A specialization of an alias template is a type alias — *end note*]. Otherwise, `false`.

```
consteval bool is_entity_proxy(info r);
```

Returns: `true` if `r` represents an entity proxy. Otherwise, `false`.

Change `dealias` to `underlying_entity_of` and add `proxied_entity_of` following the specification of `underlying_entity_of`:

```
consteval info ~~dealias~~underlying_entity_of(info r);
```

Constant When: `r` represents an entity.

Returns: A reflection representing the underlying entity of what `r` represents.

47 [*Example 1:*

```
using X = int;
using Y = X;
static_assert(dealiasunderlying_entity_of(^^int) == ^^int);
static_assert(dealiasunderlying_entity_of(^^X) == ^^int);
static_assert(dealiasunderlying_entity_of(^^Y) == ^^int);
```

— *end example*]

```
consteval info proxied_entity_of(info r);
```

* *Constant When*: `r` represents an entity proxy.

* *Returns*: A reflection representing the entity proxied by the entity proxy represented by `r`.

§ [meta.reflection.member.queries] Reflection member queries

Specify in terms of *`DEALIAS`* to disallow entity proxies from several functions. Make entity proxies *`Q`*-members-of-representable.

```
consteval vector<info> members_of(info r, access_context ctx);
```

1 *Constant When*: ~~dealias~~*`DEALIAS`*`(r)` is a reflection representing either a class type that is complete from some point in the evaluation context or a namespace.

[…]

4 A member *`M`* of a class or namespace *`Q`* is *`Q`-members-of-representable* from a point *`P`* if a *`Q`*-members-of-eligible declaration of *`M`* members-of-precedes *`P`* and *`M`* is

(4.1) — a class or enumeration type,

(4.2) — […]

(4.10) — a namespace, ~~or~~

(4.11) — a namespace alias, or

(4.12) — an entity proxy.

5 *Returns*: […]

```
consteval vector<info> bases_of(info type, access_context ctx);
```

6 *Constant When*: ~~dealias~~*`DEALIAS`*(type) represents a class type that is complete from some point in the evaluation context.

7 *Returns*: […]

```
  consteval vector<info> static_data_members_of(info type, access_context
          ctx);
```

Constant When: ~~dealias~~*DEALIAS*`(type)` represents a class type that is complete from some point in the evaluation context.

Returns: […]

```
  consteval vector<info> static_data_members_of(info type, access_context
          ctx);
```

Constant When: ~~dealias~~*DEALIAS*`(type)` represents a class type that is complete from some point in the evaluation context.

Returns: […]

```
  consteval vector<info> enumerators_of(info type_enum);
```

Constant When: ~~dealias~~*DEALIAS*`(type_enum)` represents an enumeration type and `is_enumerable_type(type_enum)` is `true`.

Returns: […]

§ [meta.reflection.layout] Reflection layout queries

Specify in terms of *`DEALIAS`* instead of `dealias` to disallow entity proxies from several functions.

```
  consteval size_t size_of(info r);
```

Constant When: ~~dealias~~*DEALIAS*`(r)` is a reflection of a type, object, value, variable of non-reference type, […]

[…]

```
  consteval size_t alignment_of(info r);
```

Constant When: ~~dealias~~*DEALIAS*`(r)` is a reflection of a type, object, variable of non-reference type, […]

Returns: […]

```
  consteval size_t bit_size_of(info r);
```

Constant When: ~~dealias~~*DEALIAS*`(r)` is a reflection of a type, object, value, variable of non-reference type, […]

§ [meta.reflection.define.aggregate] Reflection class definition generation

Specify in terms of *`DEALIAS`* instead of `dealias` to disallow entity proxies from several functions.

```
consteval info data_member_spec(info type,
                                data_member_options options);
```

4 *Constant When*:

(4.1) — ~~dealias~~*DEALIAS*`(type)` represents either an object type or a reference type;

(4.2) — […]

§ 5.3 Poll 2b: Make `^^Using::Decl` ill-formed for *`using-declarator`*s

[Drafting note: All wording assumes P2996R12.]

§ [expr.reflect] The reflection operator

Extend paragraph 5 to state that a *`reflect-expression`* is ill-formed when it names a *`using-declarator`*.

5 If a *`reflect-expression R`* matches the form `^^` *`qualified-reflection-name`*, it is interpreted as such and its representation is determined as follows:

(5.*) — If the *`identifier`* names a *`using-declarator`* ([namespace.udecl]), *`R`* is ill-formed.

(5.1) — Otherwise, ~~I~~if the *`identifier`* is a *`namespace-name`* that names a namespace alias ([namespace.alias]), *`R`* represents that namespace alias. For any other *`namespace-name`*, *`R`* represents the denoted namespace.

(5.2) — […]

§ 6 References

[P2996R10] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2025-02-27. Reflection for C++26.
https://wg21.link/p2996r10

[P2996R12] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2025. Reflection for C++26.
https://ww21.link/p2996r12

[P2996R7] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2024-10-13. Reflection for C++26.
https://wg21.link/p2996r7

[P3547R1] Dan Katz, Ville Voutilainen. 2025-02-09. Modeling Access Control With Reflection.
https://wg21.link/p3547r1